DBB-ECC: Random Double Bit and Burst Error Correction Code for HBM3

Chaehyeon Shin, *Graduate Student Member, IEEE*, and Jongsun Park, *Senior Member, IEEE*

***Abstract*—As dynamic random access memory (DRAM) technology continues to scale down, DRAM vendors have adopted on-die error correction codes (on-die ECC) to address reliability problems caused by cell failures. For burst error correction, a single symbol correction (SSC) Reed–Solomon (RS) code is utilized in high bandwidth memory (HBM) 3. However, randomly scattered errors frequently occur with aggressive technology scaling, which necessitates more robust error correction codes (ECC) scheme that addresses both burst errors and scattered errors. This brief presents double bit and burst ECC (DBB-ECC), an efficient scheme designed to correct both single symbol errors and random double bit errors with reduced implementation overhead. In the proposed decoding, syndromes based on SSC RS codes are used to address both error types without increasing parity bits. The decoder complexity has been also reduced by exploiting the syndrome patterns of double bit errors. The experimental results show that the proposed solution needs lower implementation overhead than conventional ones while maintaining same level of correction capability. Compared to the conventional SSC code, it also significantly enhances HBM3 reliability without increasing storage overhead.**

***Index Terms*—Double error correction (DEC), error correction codes (ECC), high bandwidth memory (HBM), on-die error correction codes (on-die ECC), Reed–Solomon (RS) codes, single symbol correction (SSC).**

I. Introduction

While technology scaling offers performance and power benefits for dynamic random access memory (DRAM), it makes memory cells more vulnerable to noise and variations, leading to reliability problems with rising bit error rates [1], [2]. To address the reliability issue, DRAM vendors have employed on-die error correction codes (on-die ECC) [3], which ensures that errors in data are internally corrected or detected before they are sent out of DRAM chip. At the cost of the reliability benefits, on-die ECC entails hardware overheads in terms of redundant storage for storing parity bits, with increasing delay and area for additional encoding/decoding process.

High bandwidth memory (HBM), which is widely adopted by key system-level platforms, also employs on-die ECC, where a single symbol correction (SSC) code is utilized in HBM3 [4], [5]. As bit error rate increases due to continuous technology scaling, randomly scattered faults also become more frequent in DRAMs [6], [7]. However, the error correction capability of SSC code in HBM3 is limited to the errors within a single 16-bit symbol and it cannot address randomly scattered errors. This limitation becomes a critical issue when scattered scaling faults are increasing [8]. Therefore, a robust error correction codes (ECC) scheme for HBM is highly required to address both burst errors and scattered errors.

A key challenge in designing robust on-die ECC lies in the inherent tradeoff between error correction capability and hardware implementation cost. As error correction capability increases, so does the hardware overhead in terms of parity bits and encoding/decoding complexity [9], which critically affect the cost of DRAM chips. double error correction (DEC)-SbEC [13] combines Reed–Solomon (RS) codes [12] and BCH codes [11] to correct either random double bit or single b-bit symbol errors, but it requires significant redundant storage. While Unity ECC [14] reduces this overhead by using unused syndromes from SSC decoding, its decoding method increases decoding complexity. The approach in [14] might be acceptable when implemented in the memory controller, however, the substantial decoder area is impractical for on-die ECC implementation. Moreover, existing solutions in [13] and [14] are primarily designed for an 8-bit symbol size commonly used in dual in-line modules (DIMMs), whereas HBM3 necessitates a 16-bit symbol size due to long data length. The difference in symbol size further increases the hardware implementation cost when adopting these solutions for on-die ECC in HBM3. Few studies address the substantial hardware cost of ECC scheme capable of correcting both burst and scattered errors, which is critically needed for HBM3.

In this brief, double bit and burst error-correction-code (DBB-ECC), a robust and low-redundancy ECC approach is presented to address both single 16-bit symbol errors and random double bit errors for HBM3. It corrects either burst or scattered errors using syndromes computed during SSC decoding based on RS codes, without increasing parity overhead. The decoding method in [14] involves the comparisons of a large number of syndromes, which demands a large look-up table in the decoder. To avoid this, the proposed decoding employs a novel method that exploits the syndrome patterns of double bit errors, thus significantly reducing the size of syndrome table. The simulation results show that DBB-ECC provides same level of error correction capability with lower hardware implementation cost than conventional solutions. Compared to the conventional SSC code that is currently adopted in HBM3, it significantly enhances HBM3 reliability without increasing storage overhead.

This brief is organized as follows. Section II introduces SSC RS codes and the on-die ECC in HBM3. Section III details the proposed solution for addressing single symbol and double bit errors, and demonstrates correction capabilities of various ECCs through simulation results. Section IV presents the implementation results. Finally, Section V gives conclusions.

II. Single Symbol Correction Reed–Solomon Codes

RS codes [12] are a subclass of nonbinary BCH codes constructed with symbols from a Galois Field $GF(q)$, where q is typically a

Received 2 October 2024; revised 20 December 2024 and 3 February 2025; accepted 19 February 2025. Date of publication 21 February 2025; date of current version 21 July 2025. This work was supported in part by the Institute of Information Communications Technology Planning and Evaluation (IITP) Grant funded by the Korea Government (MSIT) under Grant RS-2023-00229028; and in part by the National Research Foundation of Korea (NRF) Grant funded by the Korea Government (MSIT) under Grant RS-2024-00345481 and Grant RS-2024-00405495. This article was recommended by Associate Editor W. Liu. *(Corresponding author: Jongsun Park.)*

Chaehyeon Shin was with the School of Electrical Engineering, Korea University, Seoul 02841, South Korea. She is now with the DRAM Design Group, SK Hynix Inc., Icheon 17336, South Korea (e-mail: chaehyeon.shin@sk.com).

Jongsun Park is with the School of Electrical Engineering, Korea University, Seoul 02841, South Korea (e-mail: jongsun@korea.ac.kr).

Digital Object Identifier 10.1109/TCAD.2025.3544964

1937-4151 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Texas A M University. Downloaded on September 17,2025 at 17:52:24 UTC from IEEE Xplore. Restrictions apply.

power of two. For $q = 2^m$, m-bit symbols are used to construct the code and m is referred to as the symbol size. A *symbol error* is a single bit error (SBE) or multiple bit errors belonging to the same symbol. RS codes can correct t symbol errors with $2t$ parity symbols (e.g., 2 parity symbols are needed for SSC). The concatenated k data symbols and $2t$ parity symbols are called n codeword symbols.

A parity check matrix, which is called H-matrix, dictates the structure of the encoder/decoder. An extension for the SSC RS (n, k) code has the H-matrix which can be expressed as following:

$$H = \begin{pmatrix} 1 & 1 & \cdots & 1 & 1 & 1 & 0 \\ T_0 & T_1 & \cdots & T_{k-2} & T_{k-1} & 0 & 1 \end{pmatrix} \tag{1}$$

where $T_0, T_1, \ldots, T_{k-1}$ are different elements in $GF(2^m)$. When constructing the H-matrix, k elements $(T_0, T_1, \ldots, T_{k-1})$ are selected from $GF(2^m)$ such that each syndrome of target errors is unique. The syndrome symbols are calculated by multiplying the $2 \times n$ parity check matrix with n codeword symbols, and they are used for error detection and correction.

RS decoding starts with the syndrome computation using H-matrix. When the codeword has no error, the syndromes will be computed as zeros. Thus, if any of the syndromes is nonzero, an error is *detected*. When a single symbol error (SSE) occurs in the location i with error pattern E_i, the syndromes will be the form as following:

$$\begin{pmatrix} S_0 \\ S_1 \end{pmatrix} = \begin{pmatrix} E_i \\ T_i E_i \end{pmatrix} \tag{2}$$

where S_0 and S_1 are m-bit syndrome symbols, and E_i is 16-bit error symbol. Two syndromes of SSEs always have the relationship which can be expressed as following:

$$T_i S_0 \oplus S_1 = 0. \tag{3}$$

Here, $\oplus$ is the bitwise XOR operation. RS decoder computes $T_j S_0 \oplus S_1$ for $0 \leq j \leq k-1$. The computation result is zero only when $j = i$. Thus, the error location is determined when the computation result is zero. Using the error location i and the error pattern (S_0), a SSE is *corrected*.

On-die ECC based on RS16 (19,17) code has been employed in HBM3 chip [4]. A RS16 (19,17) code represents a 16-bit symbol RS code with 17 data symbols and 2 parity symbols. In other words, additional 32-bit parity should be stored on-chip to protect 272-bit data from a single 16-bit symbol error. If a cyclic redundancy check (CRC) code is used as system ECC in memory controller, the on-die SSC combined with system CRC can achieve a detection rate of over 99.99% [5]. The on-die SSC effectively corrects symbol errors and shows great detection coverage with system CRC at a redundancy rate of 11.76%.

One of the difficulties encountered with the on-die SSC code used in HBM3 is that it is not sufficient to correct randomly scattered errors. Previous ECC solutions designed to address symbol or scattered errors require significant redundant storage and decoder area, making them impractical for HBM3 application. So, in the following sections, we present DBB-ECC, an ECC scheme that efficiently corrects single symbol or random 2-bit errors with reduced implementation overhead.

III. DBB-ECC Decoding Method

In this section, the syndrome patterns of random 2-bit errors and a novel decoding method of DBB-ECC exploiting the patterns are presented. As a case study, RS16 (19,17) code has been modified to address random double bit and single 16-bit symbol errors within 272-bit data. DBB-ECC can be extended to RS codes with various symbol sizes and data lengths.

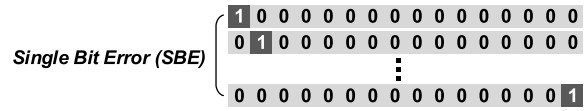
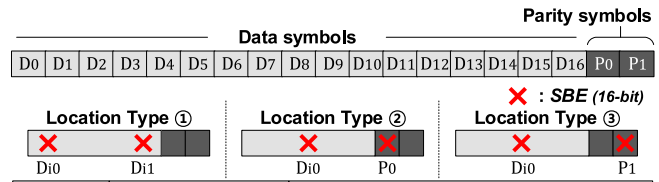


Fig. 1. SBE that occurs in a 16-bit symbol.



Location Type	Errors occurred in		Syndrome Symbols		
	Data	Parity	S0	'1' count in S0	S1
①	Di0, Di1	-	Ei0 ⊕ Ei1	0 (Ei0 = Ei1)	Ti0Ei0 ⊕ Ti1Ei1
②	Di0	P0	Ei0 ⊕ Ei1	2 (Ei0 ≠ Ei1)	Ti0Ei0
③	Di0	P1	Ei0	1	Ti0Ei0 ⊕ Ei1

Fig. 2. Three location types of random 2-bit errors in RS16 (19,17) decoding and syndrome patterns corresponding to the location types.

A. Syndrome Patterns of Random Double Bit Error

A SBE that occurs in a single 16-bit symbol is one of 16 symbols, as presented in Fig. 1. When two SBEs occur in a single symbol, it can be corrected with SSC RS codes. In contrast, when two SBEs occur in two different symbols, it is called a random 2-bit error and not correctable with SSC decoding. As presented in Fig. 2, random 2-bit errors can be classified into three types based on their location, and the syndrome patterns vary according to the location types. The codeword includes 17 data symbols $(D_0, D_1, \ldots, D_{16})$ and 2 parity symbols (P_0, P_1). Type 1 is when two SBEs occur in two data symbols (D_{i0}, D_{i1}) where $0 \leq i0 < i1 \leq 16$. There are 136 $({}_{17}C_2)$ locations for type 1. In type 2 and type 3, errors occur in one data symbol (D_{i0}) and one parity symbol (P_0 or P_1). There are 17 $({}_{17}C_1)$ locations for each type.

The syndrome patterns corresponding to the location types are illustrated in Fig. 2, where E_{i0} and E_{i1} are SBE symbols and S_0 and S_1 are 16-bit syndrome symbols. Please note that when counting the number of "1"s in 16-bit S_0 symbol, if random 2-bit error occurred, the count can only be 2, 1, or 0. In type 1 and type 2, S_0 symbol has the form of the bitwise XOR operation of two error patterns (E_{i0}, E_{i1}). Thus, when two error patterns are different ($E_{i0} \neq E_{i1}$), the count of "1"s in S_0 symbol will be 2. Otherwise, when two error patterns are same ($E_{i0} = E_{i1}$), the count of "1"s in S_0 symbol will be 0. In other words, S_0 will be zero. In type 3, S_0 symbol is same with error pattern E_{i0} in data symbol D_{i0}, and the count of "1"s in S_0 symbol is always 1. DBB-ECC corrects random double bit errors by exploiting the syndrome patterns, where the count of "1"s in S_0 symbol is always 2, 1, or 0.

B. Decoding Method

DBB-ECC utilizes RS16 (19,17) decoding for SSE correction, and it employs a novel method that uses the computed syndromes from RS decoding to efficiently correct random double bit errors without requiring additional parity or syndrome bits. The proposed double bit error (DBE) correction consists of three steps: counting the number of "1"s in S_0 symbol, estimating error patterns (E_{j0}, E_{j1}), and finding error locations $(j0, j1)$. After those steps, the error can be corrected using the decoding results. Fig. 3 show the decoding steps when the results of counting the number of "1"s in S_0 symbol are 2, 1, and 0. The details in step 2 and step 3 vary based on the results in step 1.

When the S_0 symbol has two "1"s, the location type 1 and 2 can be the candidates with different error patterns $(E_{j0} \neq E_{j1})$. For step

Authorized licensed use limited to: Texas A M University. Downloaded on September 17,2025 at 17:52:24 UTC from IEEE Xplore. Restrictions apply.

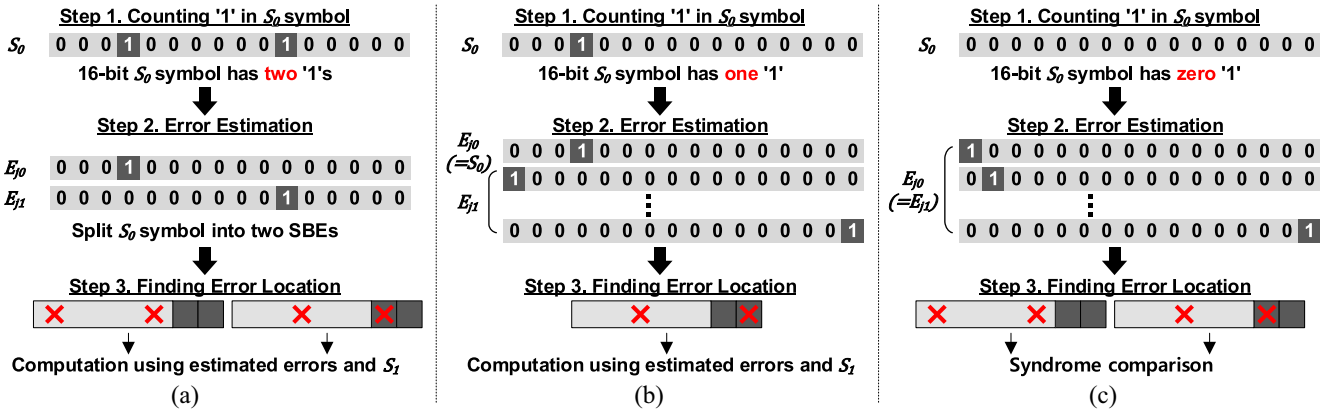


Fig. 3. Random double bit error decoding process of DBB-ECC when the count of "1"s in S_0 symbol is (a) 2, (b) 1, (c) 0.

2, the S_0 symbol is split into two SBE symbols such that the bitwise XOR of two estimated error patterns matches the S_0 symbol. Starting from the least significant bit (LSB), if a bit position in S_0 is 1, the first error pattern E_{j0} is estimated as the SBE symbol corresponding to that position, and the second error pattern E_{j1} is calculated by bitwise XOR of S_0 and E_{j0}. The two SBE symbols can be switched, so 2 pairs of error symbols (E_{j0}, E_{j1}) are considered in step 3. Then, the DBB-ECC decoder finds the error locations using the relationship between the error patterns and S_1 symbol. For type 1, the error patterns and S_1 symbol have the relationship as following:

$$T_{i0}E_{i0} \oplus T_{i1}E_{i1} \oplus S_1 = 0 \tag{4}$$

where $\oplus$ means the bitwise XOR operation, and E_{i0} and E_{i1} are actual error patterns in data symbols D_{i0}, D_{i1}. For type 2, the error patterns and S_1 symbol have the relationship as following:

$$T_{i0}E_{i0} \oplus S_1 = 0. \tag{5}$$

When the estimated error patterns satisfy these relationships, the error locations and error patterns can be determined at once.

When the S_0 symbol has one "1", the location type 3 can be the only case. For step 2, the error symbol E_{j0} in data symbol D_{j0} is estimated as S_0 symbol, and the error symbol E_{j1} in parity symbol P_1 is estimated as one of SBE symbols, so 16 pairs of error symbols (E_{j0}, E_{j1}) are considered in step 3. Then, the DBB-ECC decoder finds the error location using the relationship between the error patterns and S_1 symbol. For type 3, the error patterns and S_1 symbol have the relationship as following:

$$T_{i0}E_{i0} \oplus E_{i1} \oplus S_1 = 0 \tag{6}$$

where $\oplus$ is the bitwise XOR operation and E_{i0} and E_{i1} are actual error patterns in data symbol D_{i0} and parity symbol P_1.

When the computed S_0 symbol is zero, the location type 1 and 2 can be the candidates with identical error patterns ($E_{j0} = E_{j1}$). In this scenario, the error patterns E_{j0} and E_{j1} are estimated as same SBE symbols, regardless of the computed syndromes. Unlike cases where S_0 symbol is nonzero, a zero S_0 provides no information about the error location. Therefore, the DBB-ECC decoder finds the error location by comparing the computed S_1 symbol with stored syndromes. Errors of location type 1 and 2 with the same error patterns account for 2448 cases out of a total of 43 776 random DBE cases. DBB-ECC exploits the syndrome patterns of double bit errors to estimate error patterns, reducing the number of syndrome comparisons to only 5.6% of all DBE syndromes.

TABLE I
ERROR CORRECTION CAPABILITY AND ECC CONFIGURATION OF VARIOUS ECCS

ECC Type	Error Correction			ECC Word Configs.		
	SBE	DBE	SSE	Data bits	Parity bits	Redundancy rate
Hamming [10]	O	X	X	272	9	3.31%
BCH DEC [11]	O	O	X	272	18	6.62%
RS16 (19,17) [12]	O	X	O	272	32	11.76%
DEC-SbEC [13]	O	O	O	272	48	17.65%
Unity ECC [14]	O	O	O	272	32	11.76%
DBB-ECC	O	O	O	272	32	11.76%

C. Simulation Results

In literature, various ECCs have been presented, including Hamming [10], BCH DEC [11], RS16 (19,17) [12], DEC-SbEC [13], and Unity ECC [14]. Table I presents the error correction capabilities of different codes with SBE, DBE, and single 16-bit symbol error (SSE). It also shows the redundancy rate for 272-bit data, which is the data length used for on-die ECC in HBM3 [4]. While the redundancy rate varies depending on the data length, the relative order remains consistent. Low redundancy rate under 10% can be found in [10] and [11], but they lack the capability to correct SSEs. Reed and Solomon [12] can correct burst errors but are ineffective against randomly scattered error. 50% more parity bits are needed in [13] for additional DBE correction. Kim et al. [14] and the proposed DBB-ECC use the same number of parity bits as [12], thereby reducing redundant storage overhead while addressing both double bit errors and SSEs.

The simulations have been conducted to compare the reliability among the prior ECC schemes in HBM and the proposed DBB-ECC. Four types of error scenarios were considered: SBE, DBE, SSE, and double adjacent symbol error (DASE), which are reported as the most frequently observed errors [4]. For each scenario, ECC decoding determines the erroneous data is correctable, detectable, or undetectable (silent data correction, SDC). SDC occurs when ECC mis-corrects an uncorrectable error, resulting in undetected errors being falsely reported as corrected. The first version with on-die ECC, HBM2E, has adopted SBE correction with DBE detection for both on-die ECC and system ECC [15]. Then, the ECC scheme with on-die SSC code and system CRC code has been adopted in HBM3 [4]. The proposed DBB-ECC can be used as on-die ECC with system CRC.

Table II provides the decoding results under four error scenarios for the on-die ECC of HBM2E [15], HBM3 [4], and DBB-ECC. While the lowest number of parity bits is used, HBM2E [15] can correct SBE and most of DBE, but it cannot correct most parts of

Authorized licensed use limited to: Texas A M University. Downloaded on September 17,2025 at 17:52:24 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DECODING RESULT COMPARISON AGAINST ERROR SCENARIOS

ECC Scheme (Data, Parity)	Decoding Result (%)	Error Scenario			
		SBE	DBE	SSE	DASE
HBM2E [15] (288, 24)	Correction	100	91.9861	0.030518	0.000003
	Detection	0	8.0139	99.969482	99.661726
	SDC	0	0	0	0.338571
HBM3 [5] (272, 32)	Correction	100	4.9505	100	0
	Detection	0	95.0481	0	99.9985
	SDC	0	0.0014	0	0.0015
DBB-ECC (272, 32)	Correction	100	100	100	0.000006
	Detection	0	0	0	99.9985
	SDC	0	0	0	0.001494

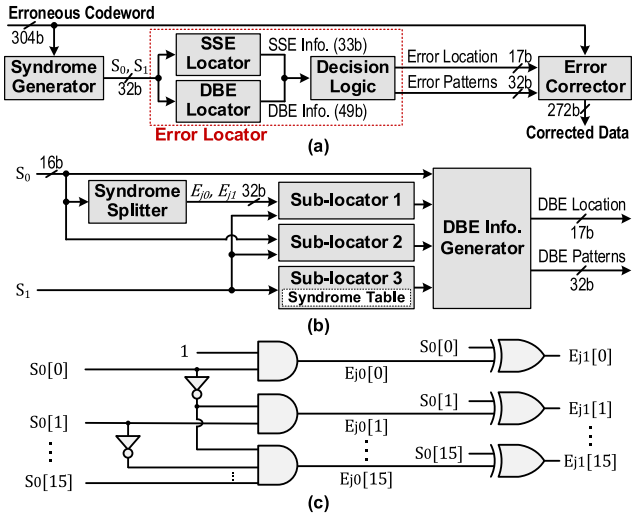


Fig. 4. (a) Block diagram of DBB-ECC decoder. The structure of (b) DBE locator and (c) syndrome splitter.

SSE and DASE. To further address burst errors, a symbol-based ECC is implemented in HBM3 [4]. It can correct SBE, SSE, and partially correct DBE that occurs within a single 16-bit symbol. However, some parts of DBE remain undetected. DBB-ECC can perfectly correct SBE, SSE, and DBE, without incurring storage overhead. Additionally, it results in a slightly lower number of undetected DASE. Overall, the proposed solute ion can improve reliability against common error scenarios compared to the solution adopted in HBM2E and HBM3.

IV. DBB-ECC DECODER IMPLEMENTATION

A. Block Diagram of the DBB-ECC Decoder

Fig. 4(a) illustrates the block diagram of the DBB-ECC decoder, which is designed to single 16-bit symbol errors or random double bit errors in a received word. The general design of RS16 (19,17) decode [16], [17], which consists of a syndrome generator, an error locator, and an error corrector, has been modified with additional modules, including a DBE locator and decision logic.

When an erroneous codeword is read from the cell array, it is first transmitted to the syndrome generator. Each codeword consists of 272-bit data and 32-bit parity. Then, the computed syndromes are sent to both the SSE locator and DBE locator, which operate in parallel. The DBE locator identifies the error locations and error patterns, as detailed in Section III. Each locator generates error information, including error locations and error patterns. The error location is represented as a 17-bit one-hot vector with the dimensions equal to the number of data symbols. For SSE, the error pattern information is 16-bit and for DBE, it is 32-bit. The decision logic selects the valid error information from the two locators and forwards it to the error corrector. Finally, the error corrector eliminates the error using XOR operations between the erroneous data symbols and identified error patterns.

Fig. 4(b) shows the detailed structure of DBE locator, which includes a syndrome splitter, three sublocators, and a DBE information (info) generator. Sublocators 1, 2, and 3 address the error cases where the count of "1" in S_0 is 2, 1, and 0, respectively. The syndrome splitter estimates error patterns (E_{j0}, E_{j1}) from the S_0 symbol as SBE symbols, which are sent to the sublocator 1. Fig. 4(c) describes the hardware architecture of the syndrome splitter in the proposed decoder. The estimation begins with the LSB, where each bit of S_0 is checked to determine the first error pattern E_{j0}. Once E_{j0} is derived, the second error pattern E_{j1} is computed as the bitwise XOR of S_0 and E_{j0}. Then, the sublocators determine the locations either by using the relationship between error patterns and S_1 symbol or by comparing syndromes. The sublocator 1 computes $T_{j0}E_{j0} \oplus T_{j1}E_{j1} \oplus S_1$ and $T_{j0}E_{j0} \oplus S_1$ for $0 \leq j0 < j1 \leq 16$, using the estimated error patterns (T_{j0}, T_{j1}) from the syndrome splitter. The sublocator 2 computes $T_{j0}E_{j0} \oplus E_{j1} \oplus S_1$ for $0 \leq j0 < j1 \leq 16$, using S_0 symbol and SBE symbol as the estimated error patterns. When the computation result is zero, the error location is determined. In the sublocator 3, S_1 symbol is compared with the precomputed syndromes in the syndrome table. The DBE info generator determines the decoding result from sublocators based on the count of "1"s in the S_0 symbol.

B. Implementation Results

The proposed DBB-ECC decoder and the decoders of SSC [12], DEC-SbEC [13] and Unity ECC [14] have been implemented for hardware comparison. Those are first Verilog coded and synthesized using Synopsys Design Compiler using 28nm CMOS standard cell library. SSC decoder is designed to correct SSEs and the other decoders provide enhanced correction capabilities, addressing both SSEs and random double bit errors. The data length (272-bit) and symbol size (16-bit) followed those in HBM3 [4].

TABLE III
COMPARISONS OF VARIOUS ECC DECODERS

	SSC [12]	DEC-SbEC [13]	Unity ECC [14]	DBB-ECC
Parity (bit)	32	48	32	32
*Decoder Area (μm^2)	3,876	22,512	76,103	16,193
Decoder Area (GE)	7,975	46,320	156,590	33,318
Decoding Latency (ns)	1.3	2.1	1.8	2.2

*The area results are obtained using a 2.5ns clock.

Table III presents the hardware comparisons of various ECC schemes. Those are compared in terms of parity bits, decoder area, gate equivalent (GE), and decoding latency. The decoding latency represents the critical path delay of the circuit, and the area results are obtained using a 2.5ns clock period. Compared to the SSC decoder, the DBB-ECC decoder shows an area and latency increase due to the additional decoding process. However, as on-die ECC modules occupy a relatively small portion of a DRAM chip compared to cell array, increasing the decoder area would be better choice than increasing storage (parity) overhead for more robust on-die ECC. DBB-ECC requires only 32-bit parity, which is the same as SSC code and lower than DEC-SbEC code. In terms of decoding overhead, the DBB-ECC decoder exhibits slightly higher latency but has the lowest area among the decoders providing same level of correction capability. DEC-SbEC code increases parity and syndrome bits, and Unity ECC involves large syndrome table, both leading to high decoding complexity. By employing the decoding

Authorized licensed use limited to: Texas A M University. Downloaded on September 17,2025 at 17:52:24 UTC from IEEE Xplore. Restrictions apply.

method based on the syndrome pattern, only 5.6% of total random double bit syndromes need to be stored and compared, significantly reducing the look-up table size. The look-up table in the DBE locator accounts for less than 10% of the DBB-ECC decoder area, representing a minimal contribution to the overall design. Overall, the proposed DBB-ECC decoder provides same level of correction capability with minimal storage overhead, the lowest area footprint, and only a marginal increase in latency compared to the conventional decoders.

V. Conclusion

While the on-die SSC code is employed in HBM3 to improve reliability, randomly scattered faults that frequently occur with continuous technology scaling, become a large problem. This brief presents DBB-ECC, a robust, low-redundancy ECC scheme that corrects both single 16-bit symbol and random double bit errors. It uses syndromes based on RS codes for both error types to reduce storage overhead, and exploits the syndrome patterns of double bit errors to reduce decoding complexity. The proposed solution needs lower implementation cost than conventional ones while maintaining same level of correction capability. Compared to the conventional SSC code adopted in HBM3, it significantly enhances HBM3 reliability without increasing storage overhead.

Acknowledgment

The EDA tool was supported by the IC Design Education Center (IDEC), South Korea.

References

[1] G. Mayuga, Y. Sato, and M. Inoue, "Highly reliable memory architecture using adaptive combination of proactive aging-aware in-field self-repair and ECC," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 39, no. 8, pp. 1688–1698, Aug. 2020.

[2] H. K. Bae, M. J. Chung, Y.-H. Gong, and S. W. Chung, "Twin ECC: A data duplication based ECC for strong DRAM error resilience," in *Proc. Design, Autom. Test Europe Conf. Exhibition (DATE)*, Apr. 2023, pp. 1–6.

[3] J. Kim, S. Kwon, J. Noh, and D.-J. Shin, "Construction of cyclic redundancy check codes for SDDC decoding in DRAM systems," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 2, pp. 736–740, Feb. 2023.

[4] M.-J. Park et al., "A 192-Gb 12-high 896-GB/s HBM3 DRAM with a TSV auto-calibration scheme and machine-learning-based layout optimization," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 256–269, Jan. 2023.

[5] Y. Ryu et al., "A 16 GB 1024 GB/s HBM3 DRAM with source-synchronized bus design and on-die error control scheme for enhanced RAS features," *IEEE J. Solid-State Circuits*, vol. 58, no. 4, pp. 1051–1061, Apr. 2023.

[6] M. K. Qureshi, D.-H. Kim, S. Khan, P. J. Nair, and O. Mutlu, "AVATAR: A variable-retention-time (VRT) aware refresh for DRAM systems," in *Proc. 45th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Rio de Janeiro, Brazil, Jun. 2015, pp. 427–437.

[7] S.-L. Gong, J. Kim, and M. Erez, "DRAM scaling error evaluation model using various retention time," in *Proc. 47th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. Workshops (DSN-W)*, Denver, CO, USA, Jun. 2017, pp. 177–183.

[8] J. Jung and M. Erez, "Predicting future-system reliability with a component-level DRAM fault model," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchit.*, Toronto, ON, Canada, Oct. 2023, pp. 944–956.

[9] W. Li, M. Zhang, T. Gui, Z. Fang, C. Xie, and F. Wu, "Improving DRAM reliability using a high order error correction code," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 43, no. 12, pp. 4775–4785, Dec. 2024.

[10] R. W. Hamming, "Error detecting and error correcting codes," *Bell Syst. Tech. J.*, vol. 29, no. 2, pp. 147–160, Apr. 1950.

[11] R. C. Bose and D. K. Ray-Chaudhuri, "On a class of error correcting binary group codes," *Inf. Control*, vol. 3, no. 1, pp. 68–79, Mar. 1960.

[12] I. S. Reed and G. Solomon, "Polynomial Codes Over Certain Finite Fields," *J. Soc. Ind. Appl. Math.*, vol. 8, no. 2, pp. 300–304, Jun. 1960.

[13] G. Umanesan and E. Fujiwara, "Random double bit error correcting—Single b-bit byte error correcting (DEC-SbEC) codes for memory systems," *IEICE Trans. Fund.*, vol. E85-A, no. 1, pp. 273–276, Jan. 2002.

[14] D. Kim, J. Lee, W. Jung, M. Sullivan, and J. Kim, "Unity ECC: Unified memory protection against bit and chip errors," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Denver, CO, USA, Nov. 2023, pp. 1–16.

[15] K. C. Chun et al., "A 16-GB 640-GB/s HBM2E DRAM with a data-bus window extension technique and a synergetic on-die ECC scheme," *IEEE J. Solid-State Circuits*, vol. 56, no. 1, pp. 199–211, Jan. 2021.

[16] H.-J. Kang and I.-C. Park, "A high-speed and low-latency Reed-Solomon decoder based on a dual-line structure," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, Orlando, FL, USA, May 2002, pp. III-3180–III-3183.

[17] M.-A. Song, S.-Y. Kuo, and I.-F. Lan, "A low complexity design of reed solomon code algorithm for advanced RAID system," *IEEE Trans. Consum. Electron.*, vol. 53, no. 2, pp. 265–273, May 2007.

Authorized licensed use limited to: Texas A M University. Downloaded on September 17,2025 at 17:52:24 UTC from IEEE Xplore. Restrictions apply.